

ALGORITHMIQUE ET PROGRAMMATION

Pr Amadou Dahirou GUEYE

Enseignant-chercheur en Informatique, UFR STA - UAM

Contact : dahirou.gueye@uam.edu.sn

PLAN

- CHAP 1 : Introduction générale
- CHAP 2 : Les structures de contrôle
- CHAP 3 : Les tableaux
- CHAP 4 : Les sous programmes
- CHAP 5 : Les structures d'enregistrements
- CHAP 6 : Les algorithmes de tri

Objectifs

A l'issu de ce cours, l'étudiant devra être capable de:

- Elaborer des algorithmes simples dans un seul bloc
- Elaborer des programmes simples dans un seul bloc
- Manipuler des tableaux à une et 2 Dimensions
- Elaborer des algorithmes à l'aide de procédures et fonctions
- Elaborer des programmes et sous programmes
- Utiliser convenablement les structures d'enregistrements et les différents algorithmes de tri

Bref, l'étudiant doit pouvoir programmer en Pascal et Python que sont les deux langages de programmation utilisés dans ce cours.

Chapitre 1 : Introduction Générale

1 Notions d'algorithme et de programme

Supposons que l'on veuille automatiser la résolution de l'équation du second degré à solutions réelles

$$ax^2 + bx + c = 0 \quad (a \neq 0)$$

- ❑ Rendre automatique la résolution d'un problème revient à confier à cette tâche à un **ordinateur**.

Sachez, à priori, qu'un ordinateur ne sait rien faire à part ce qu'on lui a appris à faire.

- ❑ Donc, si nous décidons de lui confier la résolution de cette équation, nous sommes tenus de lui apprendre à le faire.
- ❑ En d'autres termes, il nous faut lui donner **l'ensemble des instructions** qu'il doit **exécuter** pour résoudre ce problème.
- ❑ C'est cet ensemble d'instructions que nous appellerons **programme**. Et ce programme doit être écrit dans un langage compréhensible par l'ordinateur, par exemple le langage *Pascal* ou le langage *python*.

- ❑ Mais, avant l'écriture d'un tel programme, nous devons d'abord élaborer une liste ordonnée des opérations qui, appliquées aux données du problème dans l'ordre prescrit, doivent aboutir à sa résolution.
- ❑ Cette suite d'opérations s'appelle algorithme et c'est la théorie des algorithmes que l'on nomme **algorithmique**.

Définition :

- ❑ Un **algorithme** est une **suite d'actions ordonnées en séquence** qui portent sur des **objets**. Ces objets doivent être définis de manière très précise.
- ❑ Un **programme** c'est le codage d'un algorithme dans un **langage de programmation**, et qui peut être traité par une machine donnée.

La construction d'un programme se fait par étape :

Problème → Enoncé → Algorithme → Programme → Résultats attendus

Un mot sur le langage de programmation pascal

- ❑ Turbo Pascal - Compileur et un environnement de développement intégré adapté pour le langage Pascal en cours d'exécution sur le fond le système d'exploitation MS-DOS, développé par Borland
- ❑ Principales fonctionnalités
 - Programmation : Turbo Pascal offre un éditeur de textes intuitif capable de colorer automatiquement les syntaxes et les fonctions du programme. Il autorise l'utilisation de raccourcis clavier pour les différentes commandes d'édition. Les combinaisons possibles sont affichées sur l'interface de traitement.
 - Débogage : il est possible de tester le code saisi sur Turbo Pascal. Pour ce faire, il est doté d'un moteur d'exécution pouvant effectuer cette opération en mode simultané ou pas à pas. Dans ce dernier cas, il détecte automatiquement la présence d'erreurs.
 - Compilation : ce logiciel est capable de traduire le programme saisi par l'utilisateur en un code exécutable par l'ordinateur. La finalisation de la programmation produira un fichier .exe pouvant être lancé sur d'autres systèmes d'exploitation.

Un mot sur le langage de programmation pascal

❑ Les caractéristiques de Turbo Pascal

- Créer une plate-forme pour la programmation en Pascal
- Le niveau élevé sur l'arrière-plan du système d'exploitation MS-DOS
- Rédaction sur plusieurs fenêtres en même temps
- Soutien à la programmation orientée objet
- La capacité de manipuler des graphiques bien.

Un mot sur le langage de programmation python

- ❑ Python est développé depuis 1989 par Guido van Rossum et de nombreux contributeurs bénévoles
- ❑ Depuis 2015, python est le langage le plus enseigné aux USA
- ❑ Python est un langage de programmation
 - Orienté objet
 - Classes, objets, méthodes, héritage, etc.
 - Tout est objet en Python (Y compris les chaînes de caractères, les nombres, etc.)
 - supporte l'héritage multiple et la surcharge des opérateurs
 - Multi-paradigme
 - Programmation objet mais aussi programmation impérative, fonctionnelle, etc.
 - Interprété: ce qui signifie qu'il n'y a pas de phase de compilation qui traduit le programme en langage machine mais les instructions sont traitées au fur et à mesure de leur lecture par l'interprète.

Un mot sur le langage de programmation python

❑ Python est un langage de programmation

- Dynamique: Tout peut être modifié au cours de l'exécution du programme
- Syntaxe souple
 - Faible typage, simplification commande, affectation multiple
- Multi-plateforme, libre et open-source (MacOS, Windows, Linux, Android)
- Extensible: on peut facilement l'interfacer avec des bibliothèques C existantes. On peut aussi s'en servir comme d'un langage d'extension pour des systèmes logiciels complexes.

2 Objets, Données et Programmes

2.1 Notion d'Objet

Le traitement d'un **objet** concerne la **valeur** de cet objet.

Si cette valeur ne peut pas être modifiée, nous parlons de **constante**, sinon nous parlons de **variable**.

Un objet est parfaitement défini si nous connaissons ses trois caractéristiques, à savoir :

- Son ***identificateur*** : il est représenté par une suite quelconque de caractères alphanumériques (numériques et alphabétiques sans espace) commençant obligatoirement par une lettre ou un tiret. De préférence, le nom est choisi en rapport avec le contenu de l'objet.

Exemple : DELTA, Premiere_Racine .

- Sa *valeur*: constante ou variable
- Son ***type*** : nous ne pouvons pas appliquer de traitement à la valeur d'un objet si nous ne connaissons pas son type.

❑ Un **type** est défini par un ensemble de constantes et l'ensemble des opérations que nous pouvons leur appliquer.

❑ Il y a trois grands types d'objet :

- **Booléen**

Constantes : vrai, faux

Opérations : et logique, ou logique, non logique

- **Numérique** : entier ou réel

Constantes : ensemble des entiers relatifs ou ensemble des réels

Opérations : toutes les opérations arithmétiques et trigonométriques, notamment l'addition, la soustraction, la multiplication, la division, la puissance, la division entière, la partie entière d'un réel.

- **Caractère** : c'est soit une lettre de l'alphabet latin, soit un code opération ('+', '-', '*', '/', '%', ...) ou un code de ponctuation ('.', ':', '!', ...) ou un code spécial ('&', '\$', '#', ..).

Un caractère est lié à un code numérique (par exemple, code ASCII) qui le représente en machine et qui permet d'établir une relation d'ordre entre les caractères.

2.2 Instructions élémentaires

2.2.1 Affectation

L'opération d'affectation consiste à effectuer une valeur à un objet (une variable). Elle s'écrit sous la forme :

<variable> ← <valeur> (en algo)

<variable> := <valeur> (en Pascal) ou **<variable> = <valeur>** (en python)

La valeur affectée peut être :

- une variable de même type
- une constante du type de l'identificateur
- une expression dont l'évaluation produit un résultat du type de l'identificateur.

Exemples :

U ← 5

a := 3 ;

b := a ;

Delta := b*b – 4*a*c ;

>>> age = 36

2.2.2 Instructions d'entrée et de sortie

Nous disposons d'une instruction de saisie qui permet de récupérer une valeur sur un périphérique d'entrée (le clavier), et d'une instruction d'affichage qui permet l'édition d'une valeur sur un périphérique de sortie (l'écran).

La forme générale de telles instructions est la suivante :

lire (<identificateur>) pour la saisie

ecrire (<valeur>) pour l'affichage

L'identificateur est un objet déclaré. La valeur située derrière d'affichage peut être soit : un identificateur, une constante, une expression.

Exemples :

1.

lire (b)

ecrire (delta) écrire (10)

2.

ecrire ('donnez votre nom :') lire (nom)

ecrire ('donnez votre age :') lire (age)

ecrire (nom, 'vous avez ', age, 'ans')

Exercice d'application¹

Ecrire un algorithme qui lit trois nombres entiers, calcule et affiche leur somme, leur produit et leur moyenne. _____

Programme calcul ;

Variables nbre1, nbre2, nbre3, S, P : entier ;

M : réel ;

Debut

 ecrire ('donnez trois nombres entiers :');

 lire (nbre1, nbre2, nbre3) ;

 S := nbre1 + nbre2 + nbre3 ; M := S / 3 ;

 P := nbre1 * nbre2 * nbre3 ; ecrire (S, M, P) ;

Fin

Le langage utilisé pour écrire cet algorithme s'appelle ***pseudo-code*** (ou langage algorithmique).

¹ Essayez avant de regarder la solution !

Le terme **variable** permet de définir des objets variables ; des objets de nature constante pouvaient être définis avec le terme *constante*.

Les termes **debut** et **fin** délimitent les instructions de l'algorithme.

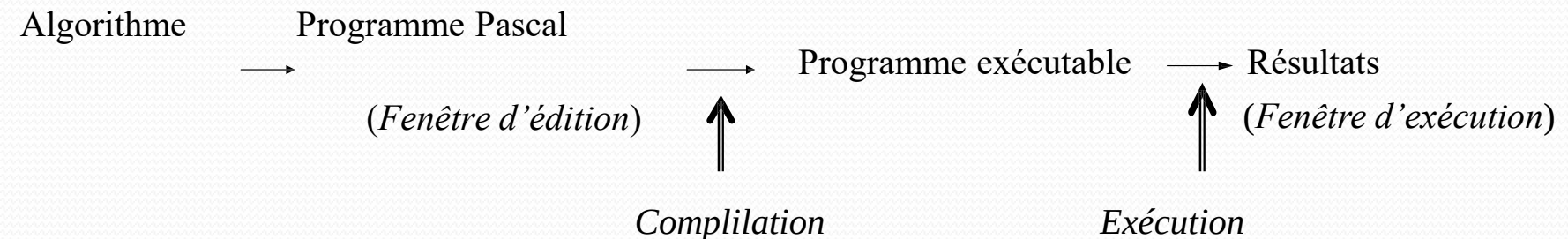
Les variables nbre1, nbre2 et nbre3 sont appelées les *données d'entrée* de l'algorithme alors que S, M et P sont appelées ses *données de sortie*.

3 Démarrer avec Turbo Pascal

Turbo Pascal est un environnement de programmation qui intègre un compilateur du langage Pascal.

Voici les différentes étapes de construction

d'exécution d'un programme en Turbo Pascal :



3.1 Structure générale d'un programme Pascal

Voici la structure générale d'un programme Pascal :

```
Program      {nom du programme} ;  
CONST       {declarations des constantes}  
TYPE        {déclaration des types} ;  
VAR         {déclaration des variables} ;  
Begin  
              {Instructions du programme principal }  
End.
```


Le point-virgule (;) permet de séparer deux instructions. Il est à noter que le compilateur du langage pascal ne fait pas la différence entre les minuscules et les majuscules.

2. Types de données élémentaires en pascal

- Le type *INTEGER* (nombres entiers) a 5 variantes :

Type	Domaine	Taille
ShortInt	-128 .. 127	1 octet
Integer	-32 768 .. 32 767	2 octets
Longint	-2147483648 ..	4 octets
	2147483647	
Byte	0 .. 255	1 octet
Word	0 .. 65 535	2 octets

- Le type *REAL* (nombres réels) a 5 variantes :

<i>Type</i>	<i>Domaine</i>	<i>Taille</i>
Real	2.9e-39 .. 1.7e38	6 octets
Single	1.5e-45 .. 3.4e38	4 octets
Double	5.0e-324 .. 1.7e308	8 octets
Extended	3.4e-4932 .. 1.1e4932	10 octets
Comp	-9.2e18 .. 9.2e18	8 octets

Le type *CHAR* (caractères) permet de représenter les caractères ASCII.

Les constantes de type caractère s'écrivent entre apostrophe : 'A', '3' ou '*'.

Le type *BOOLEAN* prend deux valeurs possibles: TRUE et FALSE.

Le type *STRING* permet de représenter les chaînes de caractères ; plus précisément des séquences, ou chaînes de caractères de longueur variable.

3.3 Déclaration des objets (constantes et variables) en Pascal

Les constantes se déclarent comme suit :

Const <identificateur constante> = <valeur> ;

Exemple :

const pi = 3.14 ;

Les variables se déclarent comme suit :

Var <identificateur variable> :

<type variable> ;

Exemple :

Var x : integer ;

Y, z: real;

3.4 Les instructions élémentaires en Pascal

L'instruction d'affectation s'écrit sous la forme

<identificateur> := <valeur> ;

3.5 Les instructions d'entrée / sortie

❑ L'instruction d'affichage s'appelle **write** et s'utilise généralement comme suit :

write (<identificateur>) ;
ou **write** (<valeur>) ;
ou **write** ('<message>') ;
ou encore **write** ('<message>', <identificateur>) ;

Le message est une constante chaîne de caractères non analysée par le compilateur et restituée comme telle à l'écran lors de l'exécution de l'instruction.

L'identificateur est remplacé par la valeur de la variable correspondante au moment de l'exécution de l'instruction.

La valeur peut être une constante ou une expression.

Exemples :

```
write('Bienvenue aux nouveaux bacheliers !') ;  
write(10) ;  
write(50+25) ;  
write('la valeur de delta est',delta) ;
```

Lorsque l'on veut aller à la ligne après un affichage, on utilise **writeln** à la place de write.

❑ L'instruction de saisie s'appelle **read** et s'utilise généralement comme suit :

read (<identificateur>) ;

Elle permet de récupérer une valeur à partir du clavier et de l'affecter à la variable désignée par l'identificateur.

Il est possible de lire plusieurs valeurs en même temps :

read (<identificateur1>, <identificateur2>, ..., <identificateurN>) ;

Il est conseillé d'utiliser **readln** à la place de read afin de supprimer le retour chariot (récupéré suite à la validation de l'utilisateur par la touche «Entrée») du tampon.

Exemples :

readln(a) ; readln(b,c) ;

3.6 Un premier programme Pascal

On va rédiger le programme correspondant à l'algorithme qui lit trois nombres entiers, calcule et affiche leur somme, leur produit et leur moyenne.

```
Program calcul ; { cette ligne est optionnelle en Turbo Pascal }
```

```
Var nbre1, nbre2, nbre3, S, P : integer ; M : real ;
```

```
begin
```

```
Write ('donnez trois nombres entiers :') ;
```

```
readln (nbre1, nbre2, nbre3) ;
```

```
S := nbre1 + nbre2 + nbre3 ;
```

```
M := S / 3 ;
```

```
P := nbre1 * nbre2 * nbre3 ; writeln (S, M, P) ;
```

```
end.
```

Remarque:

Pascal permet d'inclure des commentaires personnels à un programme ; ceci est très important pour la lisibilité et la compréhension des programmes.

Il suffit alors de mettre le texte concerné entre { mettre le commentaire ici } ou (* mettre le commentaire ici *)

4 Exercices proposés en pascal:

Exercice 1 :

Ecrire les déclarations en Turbo Pascal pour :

- a) une variable entière nommée nombre et deux variables réelles nommées rayon et surface.
- b) x,y,z : réelles, n : entière,
car, lettre : caractères, valable, ok : booléens.

Exercice 2 :

Choisir les identificateurs appropriés et écrire la partie déclaration d'un programme Pascal de facturation qui aurait :

- en donnée d'entrée : un prix hors taxes et une quantité,
- en résultat : un prix TTC

Exercice 3 :

Quels résultats produira ce programme ?

Var val, double : **integer** ;

Begin

val := 231 ; double := val*2 ;

Write(val) ;

Write(double) ;

End.

Exercice 4 :

- a) écrire un programme qui calcule et écrit le carré de 547.
- b) écrire un programme qui calcule et écrit le carré d'un entier x lu au clavier.

Exercice 5 :

Ecrire un programme qui, à partir d'un prix unitaire et d'un nombre d'articles fournis en données, calcule le prix hors taxes et le prix TTC correspondant. Le taux de TVA sera supposé égal à 18,6% et l'affichage devra se présenter ainsi :

Prix unitaire : 45.65 Nombre d'articles : 17 Prix hors taxe : 776.05 Tva : 144.35
prix TTC : 920.40

5. Syntaxe, Variables et Types de données en langage python

Numpy

- NumPy (<http://www.numpy.org/>) est un module de python qui nous permet de manipuler des tableaux ou matrices. Elle nous permet aussi d'effectuer des calculs numériques et de manipuler les fonctions mathématiques.
- Cependant dans cette partie du cours, on va seulement découvrir la partie qui concerne la computer vision, c'est-à-dire les fonctions qui nous permettent de manipuler des images
- Editeur utilisé dans cette partie: jupyterlab : facile d'utilisation pour manipuler les tableaux et plus rapide. Permet d'espacer les codes.
- Pour installer jupyterlab:
 - `pip install jupyterlab` # ou `notebook`
- Pour installer numpy:
 - `pip install numpy`
- Rq: numpy fait partie des dépendances de opencv
- Pour imposer une version
 - `pip install numpy==XXX`

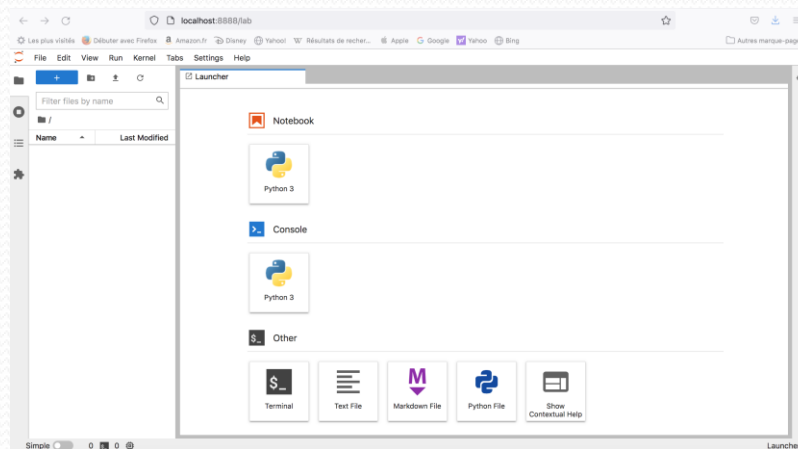
Editeur Jupyter

- Pour installer jupyterlab:

pip install jupyterlab

- Pour démarrer l'éditeur jupyterlab, on lance la commande *jupyter-lab*

- On crée d'abord un dossier TPNUMPY
- C:\Users\Da\Desktop> **mkdir TPNUMPY**
- Puis on se positionne dans le dossier et on lance la commande:
jupyter-lab pour démarrer l'éditeur



- On crée après un fichier notebook : ficda1.ipynb

5.1 Les commentaires

- Les commentaires ne sont pas exécutés par Python, ils sont destinés aux humains et non à l'ordinateur
- Ils facilitent la lecture et la compréhension du programme par d'autres personnes (ou par soi-même quelques années plus tard !)
- Commentaires simples (mono lignes)
 - Les commentaires commencent par un dièse #
 - Ils se prolongent jusqu'à la fin de la ligne
- # Ceci est mon premier commentaire !
- Commentaires longs
 - Il n'y a pas de commentaire long (type /* */) à proprement parler en Python
 - Commenter chaque ligne
 - Utiliser une chaîne de caractères longue :
 - ```
"""Ceci est une chaîne longue non utilisée,
 qui fait office de commentaire multi-ligne !
 """
```

## 5.2 Afficher à l'écran

- La fonction `print()` permet d'écrire à l'écran (dans le terminal)

```
>>> print(5 + 2)
```

```
7
```

```
>>> print("Bonjour")
```

```
Bonjour
```

```
>>> print("Bonjour", "Master SI/SR")
```

```
Bonjour Master SI/SR
```

- Rq: double quote `"` et code simple `'` sont les mêmes
- En ligne de commande, le `print()` peut être omis

```
>>> 5 + 2
```

```
7
```

# Print avec format

- La méthode `format()` a été ajoutée dans Python(2.6). La méthode de formatage des chaînes de caractères nécessite plus d'efforts manuels. Les utilisateurs utilisent `{}` pour marquer l'endroit où une variable sera substituée et peuvent fournir des directives de formatage détaillées, mais l'utilisateur doit également fournir les informations à formater.
  - Les valeurs sont soit explicitement fournies, soit référencées par l'ordre dans lequel elles apparaissent dans la procédure `format()`. Les chaînes `f` permettent d'utiliser des accolades et le préfixe `f` pour intégrer des expressions à l'intérieur de chaînes littérales. Les expressions des chaînes `f` sont évaluées et leurs valeurs appropriées leur sont substituées.
- ```
>>> print('{} a considérablement changé {}'.format('UAM', 'Diamniadio'))
```
 - UAM a considérablement changé Diamniadio!
 - ```
>>> print('{0} a considérablement changé {1}'.format('UAM', 'Diamniadio'))
```
  - UAM a considérablement changé Diamniadio!
  - ```
>>> print('{1} a considérablement changé {0}'.format('UAM', 'Diamniadio'))
```
 - UAM a considérablement changé Diamniadio!
 - ```
>>> print(f"Jaime {'Diamniadio'} par \"{'UAM'}!\")
```
  - Jaime Diamniadio par "UAM!"
  - ```
>>>
```

Aide

- Plusieurs commandes permettent d'obtenir de l'aide en Python
 - Aide sur une classe ou une méthode :
`help(list)`
`help(list.append)`
 - Lister les méthodes d'un objet :
`dir(objet)`
- Aide en ligne
 - <https://docs.python.org/3/>
 - Aussi disponible en téléchargement ou en paquet Linux

5.3 Les types de données

Votre programme !

Modules Python

Tableaux :
Module **numpy**
(utilisation rare)

Classes et objets

Type de données de collection :

Séquences :

liste (**list**)

tuple (**tuple**)

ensemble (**set**)

ensemble fixe (**frozenset**)

Tableaux associatifs (**hash**) :

dictionnaire (**dict**)

dictionnaire ordonné

(**OrderedDict**)

Type de données de base :

Nombres :

booléen (**bool**)

entier (**int**)

flottant (**float**)

Textes :

chaîne de

caractères (**str**)

chaîne binaire (**bytes**)

Fonction

Fichier

(**file**)

Nombres entiers

- ```
>>> print(1)
1
>>> print(1 + 2)
3
>>> print((3 + 4) * (5 + 8))
91
```
- Pas de limite aux nombres entiers en Python !
- ```
>>> print(123456789123456789123456789)
123456789123456789123456789
```

Nombres à virgule

- ```
>>> print(1.3)
```

```
1.3
```

```
>>> print(1.3e6)
```

```
1300000.0
```
- La précision du flottant Python correspond en fait à un double.

# Booléens

- Deux valeurs possibles : vrai ou faux
  - ◆ True
  - ◆ False

# 5.4 Variables

- Une variable est un nom auquel on associe une valeur
  - ◆ **La valeur est souvent connu seulement à l'exécution du programme (par exemple c'est le résultat d'un calcul)**
- Sous Python, les noms de variables doivent en outre obéir à quelques règles simples :
  - ◆ **Un nom de variable est une séquence de lettres ( $a \rightarrow z, A \rightarrow Z$ ) et de chiffres ( $0 \rightarrow 9$ ), qui doit toujours commencer par une lettre.**
  - ◆ **Seules les lettres ordinaires sont autorisées. Les lettres accentuées, les cédilles, les espaces, les caractères spéciaux tels que \$, #, @, etc. sont interdits, à l'exception du caractère \_ (souligné).**
  - ◆ **Attention majuscules et minuscules sont différenciées !**
  - ◆ **Il est important de trouver des noms parlants**

# Variables

- En plus de ces règles, il faut encore ajouter que vous ne pouvez pas utiliser comme nom de variables les 33 « mots réservés » ci-dessous

|      |          |        |        |       |          |        |
|------|----------|--------|--------|-------|----------|--------|
| and  | as       | assert | break  | class | continue | def    |
| del  | elif     | else   | except | False | finally  | for    |
| from | global   | if     | import | in    | is       | lambda |
| None | nonlocal | not    | or     | pass  | raise    | return |
| True | try      | while  | with   | yield |          |        |

# 5.5 Affectations simples de variables

- On crée une variable en lui donnant une valeur, avec =

```
>>> age = 36
```

- On obtient la valeur d'une variable

- En donnant son nom

```
>>> age
```

```
36
```

Ou bien avec **print**

```
>>> print(age)
```

```
36
```

- La valeur de la variable peut être modifiée avec =

```
>>> age = 37
```

```
>>> age = age + 1 # Anniversaire
```

```
37
```

- **del** permet de supprimer une variable. **del age**

# Affectations multiples

- Sous Python, on peut assigner une valeur à plusieurs variables simultanément

```
>>> x = y = 2020
[>>> x
2020
[>>> y
2020
>>> █
```

- On peut aussi effectuer des affectations parallèles à l'aide d'un seul opérateur :

```
[>>> note1, note2 = 13, 18.77
[>>> note1
13
[>>> note2
18.77
>>> █
```

# 5.6 Types de données

- Python peut manipuler différents types de données :
  - ◆ des nombres entiers (integer en anglais, abrégé en int),
  - ◆ des nombres à virgule (souvent appelé flottants ou float en anglais),
  - ◆ des chaînes de caractères (string en anglais, abrégé en str)
  - ◆ et des booléens (valeur vraie ou fausse, bool) :
- age = 38 # Entier
- poids = 76.5 # Flottant
- nom = "Dahirou Gueye" # Chaîne de caractères
- enseignant = TRUE # Booléen
- etudiant = FALSE # Booléen
- telephone = "00221 77 501 25 35" # Chaîne de caractères!



# 5.7 Opérations sur les nombres

- Opérations basique :

- ◆ Addition : +

- ◆ Soustraction : -

- ◆ Multiplication : \*

- ◆ Division : /

- ◆ Division entière : //

- ◆ Modulo (reste de la division) : %

- ◆ Puissance : \*\*

- D'autres opérations sont définies dans le module **math**

# 5.8 Priorité des opérations

- Sous Python, les règles de priorité sont les mêmes que celles qui vous ont été enseignées au cours de mathématique. Vous pouvez les mémoriser à l'aide de l'acronyme PEMDAS :

- **P pour parenthèses**

Ainsi  $2*(3-1) = 4$  , et  $(1+1)**(5-2) = 8$ .

- **E pour exposants**

Ainsi  $2**1+1 = 3$  (et non 4), et  $3*1**10 = 3$  (et non 59049 !).

- **M et D pour multiplication et division, qui ont la même priorité**

Si deux opérateurs ont la même priorité, l'évaluation est effectuée de gauche à droite. Ainsi dans l'expression  $59*100/60$ , la multiplication est effectuée en premier, et la machine doit donc ensuite effectuer  $5900/60$ , ce qui donne 98.

- **A et S l'addition A et la soustraction S**

Ainsi  $2*3-1 = 5$  (plutôt que 4), et  $2/3-1 = -1$  (Rappelez-vous que par défaut Python effectue une division entière).

# 5.9 Chaînes de caractères

- Les chaînes de caractère s'écrivent entre guillemets (simples ou doubles)
  - > `Structure="Département d'Informatique TIC"`
  - > `chaine_vide= " "`
- Les chaînes de caractère sont en Unicode
  - Cela permet de gérer toutes les langues du monde et des symboles
- Pas de type caractère en Python
  - On utilise une chaîne d'un seul caractère
- Caractères spéciaux
  - Retour à la ligne : `"\n"`
  - Tabulation : `"\t"`
  - Antislash : `"\\"`

# Chaînes de caractères

- Chaînes avec des guillemets à l'intérieur
  - « Antislasher » les guillemets à l'intérieur de la chaîne : "Il se nomme \"Dahirou\"."
  - Tripler les guillemets extérieurs : """"Il se nomme "Dahirou"."""
  - Alternier guillemets doubles et simples : 'Il se nomme "Dahirou".'

# Chaînes de caractères

## ■ Opérations sur les chaînes

- ◆ Obtenir la longueur d'une chaîne (= le nombre de caractères)
- ◆ Obtenir un caractère de la chaîne
- ◆ Obtenir une partie de la chaîne
- ◆ Concaténer deux chaînes
- ◆ Rechercher si une chaîne est incluse dans une autre

## ● Exemples avec `s = "Bonjour"`

`len(s)`       $\Rightarrow 7$

`s[0]`       $\Rightarrow \text{"B"}$       `s[-1]`  $\Rightarrow \text{"r"}$

`s[0:3]`       $\Rightarrow \text{"Bon"}$

`"Hi " + s`

`s.find("jour")`  $\Rightarrow 3$  # Trouvé en position 3  
(-1 si pas trouvé)

# Chaînes de caractères

## ■ Opérations sur les chaînes

- ◆ Passer une chaîne en minuscule

ou en majuscule

•  
`s.lower()`

`s.upper()`

- ◆ Remplacer un morceau d'une chaîne

`s.replace("Bonjour", "Good morning")`

- ◆ Demander à l'utilisateur de saisir une chaîne

`saisie = input("Entrez un nom : ")`

# Chaînes de caractères

- Formatage de chaînes de caractère : inclure des variables dans une chaîne : %

```
>>> nom = "Gueye "
```

```
>>> prenom = "Dahirou"
```

```
>>> "Salut %s !" % prenom
Salut Dahirou !
```

```
>>> "Bonjour %s %s !" % (prenom, nom)
```

```
>>> "Taux de réussite : %s %" % 100
```

```
Taux de réussite : 100 %
```

- Attention aux faux nombres !

```
telephone1 = "00221 33 823 37 38"
```

```
telephone2 = 00221338233738
```

- Quel est le type de donnée des deux variables ci-dessus ?
- Pourquoi a-t-on choisi ce type de donnée pour le premier cas?

# 5.10 Conversions

■ Il est souvent nécessaire de convertir un type de données vers un autre

- Les fonctions `int()`, `float()`, `bool()` et `str()` permettent de convertir une valeur vers un entier, un flottant et une chaîne de caractère

- `int("8")`     $\Rightarrow$  8  
   `str(8)`      $\Rightarrow$  "8"

- `input()` retourne toujours une chaîne de caractères ; il faut penser à la transformer en entier ou en flottant si l'on demande la saisie d'un nombre !

```
age = int(input("Entrez votre âge : "))
```

```
poids = float(input("Entrez votre poids : "))
```



# 5.11 Exercices en python

- Testez les lignes d'instructions suivantes. Décrivez ce qui se passe :
  - `>>> r , pi = 12, 3.14159`
  - `>>> s = pi * r**2`
  - `>>> print s`
  - `>>> print type(r), type(pi), type(s)`
  - `>>>`
- Quelle est, à votre avis, l'utilité de la fonction `type()` ?
- (Note : les fonctions seront décrites en détail, plus loin dans ce cours).